



CS H Practice Set

Writer: Hans 10-1

Editor: Jingzhe 10-8

Range: Network, Database, Machine Learning, Recursion

Note: (Review Guide/Review Practices/Practice Set) are UNOFFICIAL sources of reference and do NOT include or hint at questions in the actual exam!

1 Concepts

1. What is the primary purpose of the KNN algorithm?
 - A) To predict continuous values
 - B) To classify data based on the majority vote of its K nearest neighbors
 - C) To reduce dimensionality of data
 - D) To optimize neural networks
 - E) To generate random data
2. In KNN, how is the value of K typically chosen?
 - A) Randomly
 - B) By always setting $K=1$
 - C) Through cross-validation and balancing noise sensitivity with pattern capture
 - D) By using the largest possible value
 - E) By ignoring the dataset characteristics
3. What is a "feature" in machine learning classification tasks?
 - A) The target output of the model
 - B) An attribute or variable describing the input data
 - C) A type of neural network layer
 - D) A measure of model accuracy
 - E) A preprocessing step
4. Which algorithm is designed to maximize the separation margin between classes in a dataset?
 - A) KNN
 - B) Decision Trees
 - C) SVM Support Vector Machine
 - D) Linear Regression
 - E) Random Forest
5. In supervised learning, what is the role of labeled data?
 - A) To test the model's accuracy
 - B) To train the model by providing input-output pairs
 - C) To reduce computational complexity
 - D) To visualize data clusters
 - E) To generate synthetic data

6. What is the Euclidean distance used for in KNN?
 - A) To normalize data
 - B) To measure the similarity between data points
 - C) To reduce overfitting
 - D) To initialize weights in neural networks
 - E) To generate random numbers
7. What is the main risk of a recursive function without a proper base case?
 - A) It will return incorrect results
 - B) It will cause a stack overflow
 - C) It will run faster
 - D) It will skip iterations
 - E) It will delete data
8. Which data structure is used to manage function calls in recursion?
 - A) Queue
 - B) Stack
 - C) Linked List
 - D) Array
 - E) Tree
9. What is the primary purpose of a DBMS (database management system)?
 - A) To create the solution for a CS project
 - B) To store, retrieve, and manage structured data efficiently
 - C) To design user interfaces
 - D) To generate PA resources efficiently
 - E) To generate random numbers
10. In a relational database, what is a "foreign key"?
 - A) A field that uniquely identifies a record in its own table
 - B) A field that links to the primary key of another table
 - C) A temporary key used during data entry
 - D) A key that encrypts data
 - E) A randomly generated identifier
11. Which SQL command is used to retrieve data from a database?
 - A) TAKE

- B) GET
 - C) RETRIEVE
 - D) SELECT
 - E) FROM
12. What is the main advantage of subnetting in IP networks?
- A) It increases the total number of available IP addresses
 - B) It improves IP address utilization and network management
 - C) It reduces internet speed
 - D) It eliminates the need for routers
 - E) It randomizes IP assignments
13. What is the purpose of the tracert command?
- A) To test internet speed
 - B) To display the route packets take to a destination
 - C) To encrypt data transmissions
 - D) To assign IP addresses dynamically
 - E) To block network traffic
14. Which of the following is a private IP address range?
- A) 10.0.0.0 - 10.255.255.255
 - B) 200.100.50.0 - 200.100.50.255
 - C) 172.32.0.0 - 172.32.255.255
 - D) 192.169.0.0 - 192.169.255.255
 - E) 1.1.1.1 - 1.1.1.255
15. What is the primary function of computer vision?
- A) To process and interpret visual data like images and videos
 - B) To optimize database queries
 - C) To create popular Tiktok videos
 - D) To create SkyNet
 - E) To create PA resources

2 Coding (Recursion)

Problem 1: Factorial Function

The factorial of a non-negative integer n , written $n!$, is defined recursively as follows: $0! = 1$, and for $n > 0$, $n! = n \times (n - 1)!$. In this problem, you will implement a recursive function to compute factorials and examine how it works.

- (a) **Base Case:** What is the value of $0!$? What is the value of $1!$? Explain why a base case is necessary in a recursive definition or function.
- (b) **Recursive Implementation:** Complete the following Python function for factorial. Fill in the missing parts so that it implements the recursive definition of $n!$ given above.

```
def factorial(n):  
    if ???:          # base case  
        return 1  
    else:  
        return n * factorial(???)
```

- (c) **Tracing the Recursion:** Using your function, manually compute `factorial(5)`. Show the sequence of recursive calls and how the result is determined (you can list the calls and returns for each step).

Problem 2: Fibonacci Sequence

The Fibonacci sequence is defined recursively. The first values are $F(0) = 0$, $F(1) = 1$, and for $n \geq 2$, $F(n) = F(n - 1) + F(n - 2)$. Using this definition, we will implement a recursive Fibonacci function and analyze its behavior.

- (a) **Fibonacci Values:** Using the definition, compute $F(2)$, $F(3)$, and $F(4)$ (the 2nd, 3rd, and 4th Fibonacci numbers).
- (b) **Recursive Implementation:** Implement a recursive Python function `fib(n)` that returns the n th Fibonacci number. Include the appropriate base cases for $n = 0$ and $n = 1$, and the recursive formula for $n \geq 2$.
- (c) **Efficiency Consideration:** The recursive Fibonacci algorithm performs repeated calculations. Explain why the naive recursive approach is inefficient for large n . For example, when computing `fib(5)`, which Fibonacci numbers are calculated multiple times due to the recursive calls?

Problem 3: Permutations of a List

Generating all permutations of a collection is a classic use of recursion and backtracking. In this problem, you will recursively generate all permutations of a list of distinct elements.

- (a) **Example Permutations:** Write down all permutations of the list `[1, 2, 3]`. (This is the expected output of a permutation function for that input.)
- (b) **Helper Function (Insertion):** Write a helper function `insert_at_all_positions(x, perm)` that takes an element x and a list `perm` (which is one permutation of some list), and returns a list of new lists where x has been inserted in every possible position in `perm`. For example, `insert_at_all_positions(3, [1, 2])` should return `[[3,1,2], [1,3,2], [1,2,3]]`.
- (c) **Recursive Permutation Function:** Using the above helper, implement a recursive function `permute(lst)` that returns a list of all permutations of the list `lst`. *Hint:* Think recursively. Remove the first element, recursively get all permutations of the remaining list, then use the helper function to insert the removed element into every position of each smaller permutation.

Problem 4: Power Set (All Subsets)

Recursion can also generate all subsets of a set (the "power set"). In this problem, you'll list and generate subsets using recursion.

- (a) **Example Subsets:** List all subsets of the set $\{1, 2\}$, including the empty set and the full set.
- (b) **Include/Exclude Thought Experiment:** Suppose you have all subsets of $\{2, 3, \dots, n\}$ already listed. How could you use them to construct all subsets of $\{1, 2, \dots, n\}$? Describe what adding the element 1 does to the subsets of the smaller set.
- (c) **Recursive Implementation:** Implement a Python function `subsets(lst)` that returns a list of all subsets of the list `lst`. Use recursion as hinted in part (b): first get all subsets of the tail of the list, then create new subsets by adding the first element to each of those.

Problem 5: Binary Search (Divide and Conquer)

Binary search is a classic divide-and-conquer algorithm that can be implemented recursively. In a sorted list, you compare the target value to the middle element and then recursively search in the lower or upper half.

- (a) **Conceptual Understanding:** In a sorted array, how does binary search use a divide-and-conquer strategy to find a target value? Describe in a sentence or two how it decides which half of the array to search.
- (b) **Recursive Implementation:** Complete the following recursive Python implementation of binary search. The function `binary_search(arr, target)` should return the index of `target` in the list `arr` if found, or `-1` if not

found. You may use a helper function with additional parameters to track the current subarray bounds.

```
def binary_search_rec(arr, target, low, high):
    if low > high:
        return -1
    mid = (low + high) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search_rec(arr, target, ???, ???)
    else:
        return binary_search_rec(arr, target, ???, ???)

def binary_search(arr, target):
    return binary_search_rec(arr, target, 0, len(arr)-1)
```

- (c) **Dry Run:** Using your `binary_search` function, what index is returned for the target value 7 in the sorted list [1, 3, 5, 7, 9]? Also, trace the sequence of recursive calls by listing the values of `low`, `high`, and `mid` in each call while searching for 7.

Problem 6: Height of a Binary Tree

Assume we have a binary tree made up of nodes, where each node has a value and pointers to a left child and right child (which may be `None` if no child exists). The **height** of a binary tree is the number of nodes on the longest path from the root down to a leaf. (By definition, an empty tree has height 0, and a tree with just one node has height 1.)

For example, consider the following binary tree: - The root node has value 10. - The root's left child is a node with value 5. (Node 5 has no left child, and its right child is 7.) - The root's right child is a node with value 15. (Node 15 has a left child 12, and no right child.) In this tree, the longest root-to-leaf path has 3 nodes (for instance, 10 -> 5 -> 7).

- (a) Based on the example tree described above, what is the height of the tree? Explain how you arrived at that number (consider the paths 10 → 5 → 7 and 10 → 15 → 12).
- (b) Write a recursive function `height(root)` that computes the height of a binary tree with given `root` node. *Hint:* Use the fact that `height` of a node is `1 + max(height(node.left), height(node.right))`. Remember to consider the base case (what is the height of an empty tree `None`?).

Problem 7: Counting Islands in a Grid (Recursive Flood Fill)

Imagine a 2D grid (matrix) of integers where each cell is either 0 or 1. We say a cell with value 1 represents land, and 0 represents water. Adjacent land cells (horizontally or vertically adjacent) form an **island**. In this problem, you will use recursion (depth-first search) to count the number of islands in the grid.

For example, consider the grid:

1	0	0	1
1	0	1	0
0	0	1	0

In the above grid, there are three separate islands (groups of adjacent 1s).

- (a) Refer to the example grid above. Identify the locations of the three islands and explain why the answer is 3 for that grid.
- (b) Write a recursive helper function `exploreIsland(grid, r, c)` that floods (marks) all land cells connected to the cell at row r , column c . This function should:
 - Check boundaries and stop if (r, c) is out of bounds.
 - Stop if the cell at (r, c) is water (0) or already visited.
 - If the cell is land (1), mark it as visited (for example, change the 1 to a 0 to indicate it's now counted) and recursively call itself on the neighboring cells up, down, left, and right.
- (c) Using the helper from part (b), implement `count_islands(grid)` that iterates over all cells of the grid. Every time you find a cell that is land (1) that hasn't been visited yet, call `exploreIsland` on it to mark that entire island, and increment a counter. `count_islands` should return the total number of islands found in the grid.